A Plan to Improve Bundle Efficiency

Register Caching, Common Subexpression Elimination, and Instruction Scheduling

APARA C Compiler Project

June 22, 2026

1 Scope and Status

This is a **design document, not an implementation**. It is written in direct response to the bundle-efficiency gap quantified in the companion "Results and Observations" report (2.1%–22.1% of bundles doing real arithmetic, depending on the program) and answers the natural next question: what would actually close that gap, and in what order. Per explicit project direction, none of this is implemented yet — it is deliberately deferred until after the current thesis presentation, specifically because each of the three techniques below would touch the IR generator, register allocator, or bundler that all 5 992 existing verification checks were built and passed against. Section 6 states the validation discipline that applies when this work does start.

2 Attack Order, and Why

1. **Register caching** — stop reloading a variable that is still sitting, unmodified, in a register.
2. **Common subexpression elimination (CSE)** — stop recomputing an expression whose inputs have not changed since it was last computed.
3. **Instruction scheduling** — reorder what is left so the bundler's existing greedy packer has more independent instructions adjacent to each other to pack.

This order is deliberate, not arbitrary: scheduling can only pack what the instruction stream already contains, so it is the *last* lever, not the first — run on a stream still full of redundant loads and recomputed subexpressions, it would mostly just rearrange waste. Caching and CSE shrink the stream first; scheduling then works on whatever genuinely independent work is left.

3 1. Register Caching

The problem, with real evidence. `test_alu_full.c` declares `long long a = 17; long long b = 5;` once, then reads `a` and `b` in five separate expressions (`a+b`, `a-b`, `a*b`, `a/b`, `a%b`) with no write to either in between. The current compiler emits a fresh `$ld` for *every one* of those ten reads, because `ir_gen.py`'s `_load_var`/`_store_var` have no concept of "this variable's value is already in a register."

The plan. Add a per-basic-block cache, `var_cache: name -> Temp`, checked by `_load_var` and updated by `_store_var`:

Listing 1: Register caching — the rule, not yet implemented

```
function LOAD-VAR(name):
    if name in var_cache:  return var_cache[name]        // reuse, emit nothing
    t <- emit the existing IRLoadAddr + IRLoad sequence
    var_cache[name] <- t
    return t
```

```
function STORE-VAR(name, value):
    emit the existing IRStore sequence
    var_cache[name] <- value          // the just-stored value is now the cached one

function INVALIDATE-CACHE():          // called at every cache-unsafe point
    var_cache <- empty
```

`INVALIDATE-CACHE` must run at every point where a register's assumed contents can no longer be trusted: a label (multiple predecessors may disagree on what is cached — no data-flow merge analysis is in scope for this design), a function call (the callee may write any global the cache holds), and any store through a pointer (a pointer write can alias any other named variable, and pointer aliasing is not tracked at all today, so a conservative full flush is the only safe option until that changes).

4 2. Common Subexpression Elimination

The problem, with real evidence. Caching alone is not enough. `test_alu_full.c`'s own modulo synthesis (`a - (a/b)*b`, since APARA has no `%` instruction) recomputes `a/b` from scratch — even though `results[3] = a/b;` just computed the *exact same division* two C statements earlier, with `a` and `b` unchanged:

Listing 2: Confirmed in fresh mcode: `a/b` computed twice

```
/ $r25 ($i64) $r22 $r24        // results[3] = a / b
...                              // (a, b reloaded -- register caching's job)
/ $r5  ($i64) $r31 $r2          // SAME division, recomputed for a % b's synthesis
```

The same pattern, one level up, is why `matmul_n16.c` spends 49.5% of its bundles on address computation: `i*16` is computed once for `&A[i*16]` and recomputed, identically, a few instructions later for the `results[]` index.

The plan. Local value numbering, layered directly on top of the cache from Section 3: give every IR temp's *value* (not just its name) a canonical number, and look up that number before emitting a new binary operation.

Listing 3: CSE via local value numbering — the rule, not yet implemented

```
function EMIT-BINOP(op, left, right):
    key <- (op, value_number(left), value_number(right))
    if key in expr_table:  return expr_table[key]   // reuse -- emit nothing
    t <- emit the IRBinOp as today
    value_number(t) <- a fresh number
    expr_table[key] <- t
    return t
```

`expr_table` is flushed at exactly the same points as `var_cache` (Section 3) — a label, a call, or a pointer store invalidates both, for the same aliasing reason. Scoping this to one basic block (no cross-branch reasoning) is enough to catch both confirmed cases above, since neither crosses a label or a call.

5 3. Instruction Scheduling

The problem. The bundler (`_pack_bundles`) never reorders anything — it only packs instructions that are *already* adjacent and independent in the stream it is handed. Once caching and CSE remove the redundant loads and recomputations, what is left in `test_alu_full.c` is 13 genuinely independent operations (`a+b`, `a-b`, `a*b`, ... — none feeds another, since each operates

on its own operand pair); today they still end up one or two per bundle purely because of the order the source happens to list them in, not because of any real dependency.

The plan. A list-scheduling pass between codegen and the bundler: build a dependency DAG per basic block from the same read/write/memory information `bundler.py`'s `_parse_deps` already computes (no new hazard logic needed — the existing analysis is reused, not reinvented), then repeatedly emit any *ready* instruction (all of its true dependencies already scheduled), preferring the one on the longest remaining dependency chain so genuinely serial work — like `matmul_n16.c`'s address-then-load-then-dot chain — is never delayed behind independent work that could just as easily wait:

Listing 4: List scheduling — the rule, not yet implemented

```
function SCHEDULE(basic_block):
    DAG <- build dependency edges using bundler.py's existing _parse_deps
    ready <- {instructions with no unscheduled predecessor}
    output <- []
    while ready is not empty:
        pick <- instruction in ready with the longest remaining dependency chain
        output.append(pick); remove pick from ready
        for successor of pick:
            if all of successor's predecessors are scheduled:  add successor to ready
    return output                        // bundler.py runs unmodified on this
```

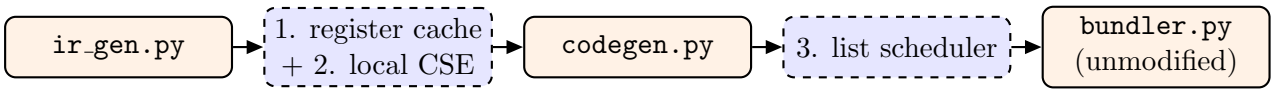

Figure 1: Where all three passes sit. Caching and CSE (1, 2) operate inside IR generation, where variable/expression identity is easiest to reason about. Scheduling (3) is a new pass between an otherwise-unmodified `codegen.py` and `bundler.py`, reusing `bundler.py`'s own dependency analysis rather than duplicating it.

6 Rollout and Validation Discipline

Each pass is implemented and verified **separately**, in the attack order above, with the full 5 992-check suite re-run after each one — not after all three together. A pass is kept only if it produces **zero new errors**; if it does produce one, the specific test and check are diagnosed before any further pass is attempted, exactly the discipline already used for every fix in this project's bug reports. Bundle counts for `matmul_n16.c` and `test_alu_full.c` are recorded before and after each pass individually, so the contribution of caching, CSE, and scheduling can be quoted separately rather than only as one combined number.

7 Expected Combined Impact

The two programs improve by different amounts for a reason worth stating plainly: `test_alu_full.c`'s 13 operations are mutually independent, so caching and scheduling together can compress it aggressively; `matmul_n16.c`'s core `$dot` computation is a real, serial chain (Section 5 of the bundling-hazards report derives exactly why), and no amount of caching, CSE, or reordering can make a truly dependent chain shorter — only the *overhead surrounding* that chain shrinks.

Table 1: Reasoned estimate, not a measurement — to be replaced with real numbers once implemented

Program	Today	After all three passes (estimate)
`test_alu_full.c`	68 bundles, 22.1% arithmetic	roughly 8–12 bundles — caching removes most of the 60.3% memory-access category, scheduling then packs the 13 now-independent operations several per bundle
`matmul_n16.c`	95 bundles, 2.1% arithmetic	address-computation category (currently 49.5%, the largest) roughly halved by CSE on `i*16/j*16`; full-loop bundle count improves more modestly, since the `$dot` chain itself is a genuine serial dependency no reordering can shorten